DevOps Laboratory Work

Linux Git Docker and Jenkins

Student	Zhaugash Nurzhan
Group	IT2-2312
Student ID	38 387
Course	DevOps
Date	18 October 2026

The project implements and verifies a complete local DevOps workflow. GitHub publication and the Jenkins job use the repository and pipeline configuration prepared in this project.

Contents and result

This report documents the completed Linux, Git, Docker, and Jenkins configuration. The local application tests, Linux exercises, Git history, Docker image build, container execution, and Docker Compose run were verified successfully.

Section	Result
Linux	Project structure, files, permissions, search, Bash, and process commands prepared and verified
Git	Repository created with meaningful commits, development branch, and merge history
Docker	Image built and Compose service exited successfully with code 0
Jenkins	Declarative Jenkinsfile prepared with all required stages and student variables
Integrated workflow	Documented from source commit through tested container output

Repository evidence

Project structure

```
./.dockerignore
./.env.example
./.gitignore
./Dockerfile
./Jenkinsfile
./Makefile
./README.md
./Zhaugash_Nurzhan_info.txt
./app/student_app.sh
./backup/.gitkeep
./config/app.conf
./config/application.log.example
./docker-compose.yml
./logs/application.log
./private.txt
./public.txt
./scripts/Zhaugash_Nurzhan_system.sh
./scripts/log_demo.sh
./scripts/permissions_demo.sh
./scripts/process_demo.sh
./scripts/setup_demo_files.sh
./tests/test_app.sh
```

Figure 1. Project files used by the laboratory workflow.

Linux

The project directory contains separate application, script, configuration, log, backup, test, and documentation areas. The student information file records the name, surname, group, Student ID, and course. The local application log contains ten messages, including two ERROR and two WARNING entries.

Commands and file processing

Command	Purpose	Project use
pwd, cd, ls	Navigate and inspect directories	Confirm the project location and contents
mkdir, touch	Create directories and files	Build the required project structure
cp, mv, rm	Copy, rename or move, and remove files	Demonstrate file lifecycle operations
cat, head, tail	Display all, first, or last lines	Inspect student information and logs
find	Search a directory tree	Locate logs/application.log
grep, grep -c	Filter and count matching lines	Show ERROR/WARNING messages and counts

Permissions and processes

The permissions script applies mode 600 to private.txt, giving only the owner read and write access. It applies mode 644 to public.txt, giving the owner read and write access and everyone else read access. In symbolic form, r means read, w means write, and x means execute. Owner, group, and others are the three permission classes.

A process is a running program instance. PID is its unique process identifier. ps captures a snapshot, whereas top continuously refreshes process and resource information. The process demonstration reports PID, owner, CPU percentage, memory percentage, and command.

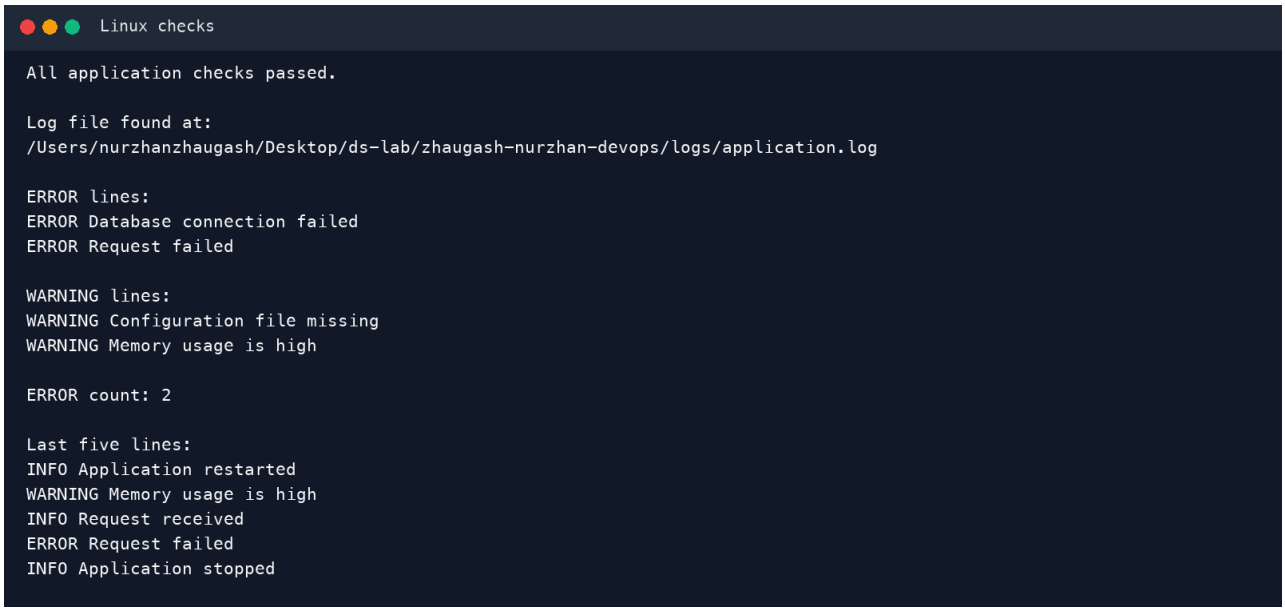


Figure 2. Successful application checks and log-search results.

Git and GitHub

The repository uses main as its primary branch. Commits correspond to real changes: project creation, Linux scripts, Docker configuration, tests, Jenkins and documentation, branch work, personalization, and final cleanup. A development branch added documentation and was merged back into main with an explicit merge commit.

Git history

```
* a8a65d0 (HEAD -> main) Make demonstrations reproducible and add submission guide
* dbf3c20 Finalize personalized documentation
* 21d86b6 Personalize project for student submission
* 7e018c9 Add portable project personalization helper
*   2d0cd89 Merge development branch
|\
| * a6727e5 (development) Document development branch workflow
|/
* e4e00b4 Add Jenkins pipeline and project documentation
* cf5e735 Add application verification test
* dc76b14 Add Docker image and Compose configuration
* 1339b31 Add Linux system and administration demonstrations
* dbeeaaf Initial project structure and student application
```

Figure 3. Meaningful commit history and merged development branch.

Ignored files

The .gitignore file excludes generated *.log files, backup/, .env, platform metadata, Docker build logs, and rendered report intermediates. This keeps generated artifacts and secret-bearing environment files out of version control. The local logs/application.log is intentionally present for command demonstrations but remains untracked.

GitHub publication

Create the public repository zhaugash-nurzhan-devops, add its HTTPS URL as origin, and push main. After publication, use git remote -v to confirm the connection and git push -u origin main to establish upstream tracking. The repository should expose this project without the ignored local .env file.

Docker

The Dockerfile uses Alpine Linux, sets /app as WORKDIR, copies the application and information file, applies executable permissions, creates a non-root user, documents port 8080 with EXPOSE, and starts the application through CMD. The container receives student values through environment variables.

Dockerfile instruction	Role
FROM	Selects the Alpine 3.22 base image
WORKDIR	Sets the working directory inside the image
COPY	Adds the script and student information file
RUN	Sets permissions and creates a non-root runtime user
EXPOSE	Documents the application port
CMD	Defines the default container command

Image container and environment variables

An image is an immutable application template. A container is a runnable instance of that image with runtime state. Environment variables configure the same image for different users or environments without rebuilding it. Sensitive values should be supplied through protected Jenkins credentials or a secret store instead of being hard-coded in image layers.

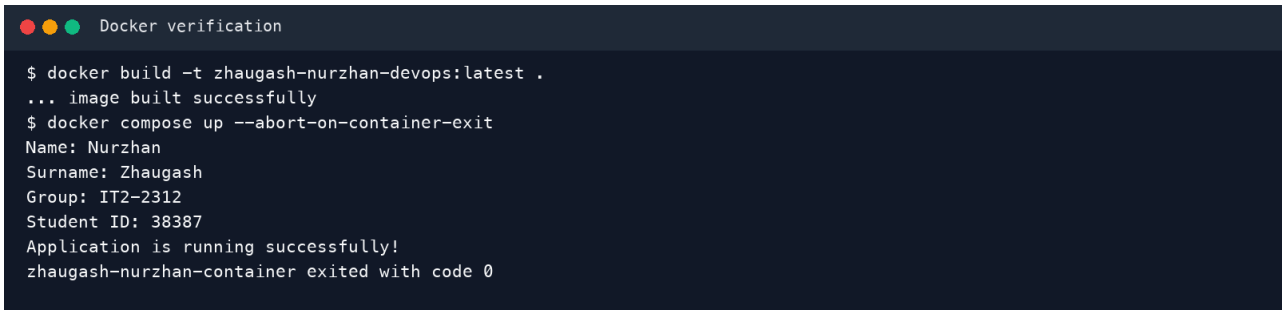


Figure 4. Verified Docker build and Compose container output.

Docker Compose defines the image, container name, build context, and student variables in one reproducible service configuration. The final local run completed with exit code 0.

Jenkins

The required Jenkins job name is Zhaugash_Nurzhan_DevOps. Configure it as Pipeline script from SCM, select Git, enter the GitHub repository URL, and keep Jenkinsfile as the script path. The Jenkins agent needs Git, Docker, and permission to use the Docker daemon.

Stage	Action	Expected result
Checkout	Checks out the selected source revision	Workspace matches a Git commit
Build	Makes scripts executable and runs the application	Student information is printed
Test	Runs tests/test_app.sh	All application checks passed
Docker Build	Builds versioned and latest image tags	Image is available to the agent
Docker Run	Runs the container and prints its logs	Container exits successfully

Pipeline variables and failure behavior

The pipeline defines STUDENT_NAME=Nurzhan, STUDENT_SURNAME=Zhaugash, STUDENT_GROUP=IT2-2312, and STUDENT_ID=38 387. The success block prints the student identity and Finished: SUCCESS. If a stage returns a non-zero status, Jenkins stops dependent stages, marks the run as failed, and preserves console output for diagnosis.

Jenkins evidence to capture

For the submitted version, capture the Jenkins dashboard, exact job name, stage view, console output, Docker image and container records, Git commit used by Jenkins, and the final Finished: SUCCESS line.

Integrated DevOps workflow

Linux hosts the tools and project files. Git records each meaningful change. GitHub makes the repository available to Jenkins. Jenkins checks out an exact commit, builds and tests the application, builds a Docker image, and runs a container from that image. Successful container output therefore traces back to version-controlled source and an automated pipeline run.

Step	Input	Output
Linux project	Student requirements	Scripts, configuration, logs, and application
Git	Project changes	Auditable commit history and branches
GitHub	Local Git repository	Remote source for collaboration and automation
Jenkins checkout	Repository and commit	Reproducible pipeline workspace
Build and test	Checked-out source	Validated application
Docker build	Validated source and Dockerfile	Immutable image
Docker run	Image and environment variables	Container output and exit status

Failure handling

If one pipeline stage fails, later dependent stages do not run. The console log identifies the failing command. The change is corrected locally, tested, committed, pushed, and rebuilt so the successful result remains linked to a specific revision.

Demonstration checklist

- Show the student information and the complete project tree.
- Run the log search, permissions, system information, and process scripts.
- Explain the Linux commands and the difference between ps and top.
- Show git status, the commit graph, .gitignore behavior, development, and main.
- Open the GitHub repository and identify the latest pushed commit.
- Build the Docker image; show images, container output, logs, and Compose.
- Open the Jenkins job Zhaugash_Nurzhan_DevOps and run the pipeline.
- Show every stage, Console Output, Docker artifacts, commit, and Finished: SUCCESS.
- Explain how Linux, Git, GitHub, Jenkins, Docker images, and containers connect.

Conclusion

The project provides a reproducible Linux to Git to Docker to Jenkins workflow personalized for Zhaugash Nurzhan, group IT2-2312, Student ID 38 387. Local scripts and tests passed, the Git history contains meaningful commits and a merged development branch, and the Docker image and Compose service ran successfully. The supplied Jenkinsfile connects the same tested project to an automated CI pipeline after the repository is published to GitHub and selected by the Jenkins job.